🎓 Guía Maestra de Demostración y Oratoria

Asignatura: Big Data / Sistemas Distribuidos — **Universidad del Valle** (LG14)
Proyecto: Despliegue y Validación de un Clúster Distribuido Apache Spark con Docker (Grupo 4)

Criterio Rector: Encontrar → Comprender → Desplegar → Probar → Analizar

⏱️ 1. Cronograma de la Presentación (7 a 10 Minutos)

Fase	Tiempo	Objetivo Principal de la Demostración
1. Introducción y Selección	1 min	Justificar la elección de Apache Spark y registro formal del repositorio.
2. Arquitectura del Clúster	2 min	Explicar los 4 contenedores, red Bridge, volumen compartido y flujo RPC.
3. Demostración en Vivo	4 min	Validar `docker ps`, Master UI (2 workers) y ejecutar las 3 pruebas funcionales.
4. Comparativa con Hadoop	2 min	Defender la tabla de 11 criterios: Spark en memoria vs MapReduce en disco.
5. Conclusiones y Cierre	1 min	Presentar historial de 5 commits y lecciones aprendidas.

🎙️ 2. Guion de Oratoria y Acciones en Pantalla

FASE 1: Introducción y Selección del Repositorio

1 Minuto

🖥️ **Acción en pantalla:** Mostrar portada del `README.md` (Sección 1 y 2).

> *"Buenas tardes, docente y compañeros. Para la práctica LG14 seleccionamos el repositorio público **spark-standalone-cluster-on-docker** de André Perez, un proyecto open-source activamente mantenido con más de 500 estrellas.*
> *Seleccionamos **Apache Spark 3.5.7** en lugar del esquema tradicional de Hadoop porque representa el estado del arte en Big Data moderno: procesamiento distribuido en memoria RAM, lo que permite consultas hasta 100 veces más rápidas que MapReduce y soporte analítico directo en Python mediante PySpark y JupyterLab."*

📌 **Punto Clave a Resaltar del Requisito:** Cumple con ser diferente al repositorio base, utiliza Docker Compose, posee arquitectura distribuida real y permite múltiples pruebas funcionales.

FASE 2: Comprensión y Análisis de la Arquitectura

2 Minutos

🖥️ **Acción en pantalla:** Mostrar el Diagrama Mermaid de Arquitectura en el `README.md` (Sección 4.3).

> *"Analizando la arquitectura técnica, nuestro entorno está compuesto por **4 contenedores Docker** orquestados en una red Bridge privada:*
> *1. **spark-master**: Nodo coordinador. Expone el puerto `7077` para RPC interno y su Web UI en el puerto `8085` para monitoreo.*
> *2. **spark-worker-1 y spark-worker-2**: Nodos ejecutores, cada uno con 1 Core de CPU y 1 GB de memoria RAM.*
> *3. **jupyterlab**: Capa cliente en el puerto `8888` para desarrollo analítico en Python sin saturar los nodos de procesamiento.*
> *4. **Volumen `hadoop-distributed-file-system`**: Montado en `/opt/workspace` en todos los contenedores para persistencia común de datasets y scripts."*

📌 **Punto Clave a Resaltar:** Los workers dependen del Master (`depends_on: [spark-master]`) y se registran dinámicamente mediante heartbeats RPC.

FASE 3: Despliegue y Demostración Práctica en Vivo

4 Minutos

1. Estado de los Contenedores en Terminal:

```
docker ps
```

> *"Podemos observar que los 4 contenedores están en estado **Up**, con sus puertos correctamente mapeados y saludables."*

2. Monitoreo en la Web UI de Spark Master:

Abrir en el navegador: `http://localhost:8085`

> *"Al ingresar a la interfaz del Master, confirmamos el estado **ALIVE** y la métrica fundamental de Big Data: **Alive Workers = 2**, con 2 Cores y 2 GB de memoria totales listos para cómputo."*

3. Ejecución del Circuito de Pruebas Distribuidas:

```
python scripts/run_all_tests.py
```

Explicación de cada prueba durante la ejecución en vivo:

- **Prueba 1 (SparkPi - Monte Carlo):** *"Dividió el cálculo de Pi en 10 particiones asignadas en paralelo a los dos workers (`172.21.0.3` y `172.21.0.4`), imprimiendo `Pi is roughly 3.14...` y demostrando paralelismo de CPU real."*
- **Prueba 2 (PySpark DataFrames):** *"Ejecuta `test_pyspark.py` aplicando transformaciones en memoria y filtrado distribuido sobre 5 registros en menos de 10 segundos."*
- **Prueba 3 (Spark SQL y Agregaciones):** *"Ejecuta `test_spark_sql.py` con consultas SQL nativas y agregaciones `groupBy` de telemetría de CPU y memoria sobre datos particionados."*

FASE 4: Análisis Comparativo Completo con docker-hadoop (11 Criterios)

2 Minutos

🖥️ **Acción en pantalla:** Mostrar la Tabla Comparativa Completa:

Criterio Oficial	docker-hadoop (Referencia Univalle)	spark-standalone-cluster (Nuestro Proyecto)
1. Tecnología Principal	Apache Hadoop (HDFS + MapReduce / YARN)	**Apache Spark (v3.5.7)**
2. Docker	Sí (Imágenes Debian/Alpine)	**Sí (Debian con SapMachine Java 21)**
3. Docker Compose	Sí (Versión 2.x/3.x)	**Sí (Orquestación con named volumes)**
4. Contenedores	5 (Namenode, Datanode, Resourcemanager, etc.)	**4 (Spark Master, Worker 1, Worker 2, JupyterLab)**
5. Almacenamiento	HDFS con replicación por bloques	**Volumen compartido `shared-workspace` (/opt/workspace)**
6. Procesamiento	MapReduce por lotes (I/O intensivo en disco)	**En Memoria RAM (RDDs / DataFrames, hasta 100x más rápido)**
7. Interfaces Web	NameNode (`9870`), YARN (`8088`), History (`8188`)	**Spark Master (`8085`), Workers (`8081/8082`), Jupyter (`8888`)**
8. Persistencia	Volúmenes dedicados NameNode/DataNodes	**Docker Named Volume persistente `hadoop-distributed-file-system`**
9. Complejidad Inst.	Media-Alta (formateo HDFS y sincronización YARN)	**Baja-Media (despliegue ágil y escalado directo de workers)**
10. Documentación	Enfocada en infraestructura básica de clúster	**Enfocada en ciencia de datos interactiva (Python, Scala, R)**
11. Caso de Uso	Batch histórico y almacenamiento masivo	**Analítica en tiempo real, streaming, SQL y Machine Learning**

> *"Conclusión: Hadoop es excelente para archivo y procesamiento por lotes en frío, pero **Spark es la herramienta líder para analítica moderna, consultas interactivas y ciencia de datos**."*

FASE 5: Conclusiones y Cumplimiento de Entrega

1 Minuto

> *"Para finalizar, el proyecto se encuentra publicado en nuestro repositorio público de GitHub **bigdata-grupo-4**, cumpliendo con la documentación técnica completa, diagramas de arquitectura, scripts de validación y un historial de **5 commits** que evidencian el avance incremental del trabajo. Quedamos a disposición para preguntas."*

🛡️ 3. Banco de Preguntas Defensivas

❓ P1: ¿Cómo demuestran que las tareas realmente se ejecutan de forma distribuida y no en un solo nodo?

Respuesta: En los logs de ejecución de `SparkPi` y `test_pyspark.py`, el `DAGScheduler` de Spark divide el trabajo en `ResultStages` con múltiples particiones. En los logs se observan los ejecutores `app-...-/0` y `app-...-/1` operando en las IPs distintas de ambos nodos (`172.21.0.3` de worker-1 y `172.21.0.4` de worker-2). Además, en la Web UI (`localhost:8085`) se ve el balanceo de Cores y memoria en ambos trabajadores.

❓ P2: ¿Qué sucede si se cae un nodo trabajador (Worker)?

Respuesta: Apache Spark posee tolerancia a fallos mediante el linaje de RDDs (Resilient Distributed Datasets). Si un worker cae, el Master redistribuye automáticamente las particiones faltantes al worker restante sin interrumpir la sesión del usuario.

❓ P3: ¿Por qué configuraron el puerto Web del Master en 8085?

Respuesta: Para evitar colisiones en entornos locales donde el puerto estándar `8080` suele estar reservado por servidores del sistema (como Oracle XE o Apache HTTP), mapeamos `8085:8080` en el host manteniendo intacto el puerto interno `8080` dentro del contenedor.

📋 4. Checklist Pre-Presentación

- [x] Docker Desktop iniciado y en verde (**Engine running**).
- [x] Contenedores arriba (`docker ps` muestra los 4 contenedores Up).
- [x] Pestaña 1 lista: Spark Master en `http://localhost:8085` (2 Alive Workers).
- [x] Pestaña 2 lista: JupyterLab en `http://localhost:8888` .
- [x] Pestaña 3 lista: `README.md` en GitHub/VSCode para mostrar arquitectura y comparativa.
- [x] Terminal abierta lista para ejecutar `python scripts/run_all_tests.py` .